Building a FFNN: a Regression Analysis on the One-Dimensional Runge Function and Study of the Classification of Handwritten Digits using the MNIST data set

Marie Océane Pilskog, Synne Sigstad Jørgensen and Muhammad Mohsin Mumtaz
(Dated: November 9, 2026)

After building a feed-forward neural network (FFNN) from scratch, we test out its capabilities for the one-dimensional Runge problem, as well as the multiclass classification exercise presented by the handwritten digits of the MNIST data set. Following an extensive k-fold cross-validation analysis, we obtain our best results for the regression of the one-dimensional Runge function using a network of five hidden layers of 100 nodes, trained for 100 epochs using stochastic gradient descent with optimiser *rmsprop*, $\gamma = 0.001$, and an ℓ_2 penalty with $\lambda = 0.0005$. On a data set of 2000 points with noise standard deviation $\sigma = 0.1$, this model delivers an MSE ≈ 0.0272. For the classification of handwritten digits in the MNIST data set, we obtain the highest accuracy, 98.26%, with a network of two hidden layers with 100 and 50 nodes, respectively, using the ReLU activation function. This network is trained with plain gradient descent, a learning rate $\gamma = 0.518$, and no regularisation ($\lambda = 0$).

I. INTRODUCTION

Neural networks are inspired by the workings of the human brain [1]. Used in fields such as image recognition, natural language processing, and speech recognition, neural networks are some of the most widely used machine learning algorithms. In simple terms, a neural network consists of layers of connected nodes, called neurons, where each neuron computes a weighted sum with a bias of its inputs and passes the result through a non-linear activation function [2]. By adjusting the weights during training, the network is able to learn how to map the inputs to the desired outputs. Neural networks can be used to solve two main kinds of problems: regression problems, where the goal is to predict a set of continuous output values, and classification problems, where inputs are assigned to a distinct class among a group of choices.

The one-dimensional Runge function, see Equation 1, will be used to illustrate the use of neural networks in regression problems. The function is often used as a test case for approximation methods. Runge discovered that when interpolating the function by polynomials on equally spaced points, using an increasing polynomial degree will not necessarily improve the approximation. On the contrary, as the degree rises, the polynomial will develop growing oscillations at the edges of the interval; a result given the name of Runge phenomenon after Runge himself [3]. The function presents as a natural benchmark for comparing polynomial-based regression methods with neural networks. The latter learn their features from the data instead of relying on a fixed polynomial basis.

$$f(x) = \frac{1}{1 + 25x^2}, \tag{1}$$

The MNIST data set of handwritten digits corresponds to 70 000 grey-scale images of 28×28 pixels, each illustrating a single handwritten digit from 0 to 9 [4]. A standard split for the data set is given by 60 000 training images and 10 000 test images. Each image is typically flattened to a 784 pixel value vector. The classification problem described by the MNIST dataset presents as a natural test of the network's ability to learn features of a provided dataset. In contrast to the Runge function, where suitable polynomial features may be chosen using classical regression methods, there is no deliberate set of features for recognising handwritten numbers.

In this analysis we develop our own feed-forward neural network (FFNN) with backpropagation, and analyse its performance on the Runge function in comparison to the ordinary least squares (OLS) results obtained in [5], using networks set up with one and two hidden layers. We further examine how the selection of the optimiser and learning rate will impact training, further comparing plain gradient descent with stochastic gradient descent using *rmsprop* and *adam*. The influence of using different activation functions in the hidden layers is studied. Moreover, the number of hidden layers and nodes is optimised, and its connection to overfitting is examined. ℓ_1 and ℓ_2 penalties are added to the cost function, and the results are compared to Lasso and Ridge regression results obtained previously in [5]. The second part of the analysis focuses on the classification of the MNIST data set. To set up our network for classification, we replace the output layer with a softmax activation function and use the cross-entropy as a cost function.

Our report is organised as follows. In section II, we aim to explain the relevant mathematical framework used in the development of our FFNN model. We derive the cost functions and their gradients, although only the MSE and softmax cross-entropy will be used in this study. We also derive the expressions and first derivatives of the four activation functions, which will be considered: sigmoid, tanh, ReLU, and Leaky ReLU. In section III, the main algorithms developed for the analysis will be presented in the form of pseudo-algorithms. Section IV presents a thorough analysis of our obtained results. Our main goal is to perform a k-fold cross-validation analysis to infer the optimal parameters for our models. The obtained final network is compared with logistic regression. This means that we compare both our optimised regression and clas-

sification networks with the network exempt from any hidden layers. A critical evaluation of the strengths and weaknesses of each method for regression and classification is given. Finally, in section V, we conclude our findings.

Our developed FFNN algorithm may be found in the following GitHub repository [1].

II. THEORY

A. Analytical derivations of the cost functions, their gradients and the ℓ_1 and ℓ_2 norms

The following derivations in this section are taken from [2, Chapter 8].

1. Notation

First, let us define the layers as $l = 0, 1, ..., L$, with $l = 0$ and $l = L$ corresponding respectively to the input and output layer. M_l describes the number of nodes found in layer l. The activation z of node j in layer l may be written as

$$z_j^l = \sum_{i=1}^{M_{l-1}} w_{ij}^l a_i^{l-1} + b_j^l, \tag{2}$$

where w_{ij}^l represents the weight linking the i-th node of the previous layer $l-1$ to the j-th node of the current layer l, while b_j^l denotes the bias term added at node j of layer l. Furthermore

$$a_j^l = f(z_j^l), \tag{3}$$

with $a_i^0 = x_i$ describing the inputs of the network. In matrix form, Equation 2 and Equation 3 are given by

$$\mathbf{z}^l = (\mathbf{W}^l)^T \mathbf{a}^{l-1} + \mathbf{b}^l, \qquad \mathbf{a}^l = f(\mathbf{z}^l), \tag{4}$$

where $\mathbf{W}^l$ is a matrix with the subweights w_{ij}^l, $\mathbf{a}^l$ is a vector with elements a_i^{l-1} and $\mathbf{b}^l$ with elements b_j^l for the layer l and the output being defined as $\mathbf{a}^L = \tilde{\mathbf{y}}$.

2. Optimisation Problem

For all weights and biases in a single parameter vector $\mathbf{\Theta} = \{\mathbf{W}^l, \mathbf{b}^l\}_{l=1}^L$, the optimisation problem is described as finding the optimal set of parameters $\hat{\mathbf{\Theta}}$

$$\hat{\mathbf{\Theta}} = \arg\min_{\mathbf{\Theta}} C(\mathbf{\Theta}), \tag{5}$$

where

$$C(\mathbf{\Theta}) = \frac{1}{n} \sum_{i=1}^{n} L(y_i, \tilde{y}_i(\mathbf{\Theta})) + \lambda \mathbf{\Omega}(\mathbf{\Theta}), \tag{6}$$

with y_i the targets the model predicts, $\tilde{y}_i$ corresponds to the predicted values, and n denotes the number of values that are being predicted. L describes the per-sample loss, $\mathbf{\Omega}$ is the regularisation term, and λ is the penalty parameter.

3. Mean Square Error

For the Mean Squared Error (MSE) without any regularisation term, let us define

$$\sum_{i=1}^{n} L(y_i, \tilde{y}_i(\mathbf{\Theta})) = ||\tilde{\mathbf{y}} - \mathbf{y}||_2^2 \tag{7}$$

with $\mathbf{a}^L = \tilde{\mathbf{y}}$ a vector with the output prediction values $\tilde{y}_i$ and $\mathbf{y}$ a vector with the target values with elements y_i, such that

$$C(\mathbf{\Theta}) = \frac{1}{2n} ||\mathbf{a}^L - \mathbf{y}||_2^2. \tag{8}$$

With backpropagation, the general formula for a given error $\boldsymbol{\delta}^l$ in the layer l is given by

$$\boldsymbol{\delta}^l = (\mathbf{W}^{l+1} \boldsymbol{\delta}^{l+1}) \circ \sigma'(\mathbf{z}^l), \tag{9}$$

where σ corresponds to the activation function of the model. This is further described in section II B.

The recursive formula Equation 9 finds the corresponding error layers $\boldsymbol{\delta}^l$, by propagating from the output layer to the first hidden layer. The output layer is defined using the chain rule as

$$\boldsymbol{\delta}^L = \sigma'(\mathbf{z}^L) \circ \frac{\partial C}{\partial \mathbf{a}^L}. \tag{10}$$

For the MSE the output error layer follows $\sigma'(\mathbf{z}^L) = 1$, since no activation function is used here, and using Equation 10 it becomes

$$\boldsymbol{\delta}^L = 2(\mathbf{a}^\mathbf{L} - \mathbf{y}). \tag{11}$$

The factor of 2 cancels out with the factor of $\frac{1}{2}$ from the cost function, giving the gradients without a regularisation term for each layer l

$$\nabla_{\mathbf{W}} C(\mathbf{\Theta}) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{a}_i^{l-1} (\boldsymbol{\delta}_i^l)^T \tag{12}$$

and

$$\nabla_{\mathbf{b}} C(\mathbf{\Theta}) = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{\delta}_i^l. \tag{13}$$

[1] https://github.com/synnesjor/FYS-STK4155/tree/main/Project2

4. ℓ_1 and ℓ_2 norms

When applying the ℓ_1 norm regularisation, we obtain an additional term

$$\mathbf{\Omega}(\mathbf{\Theta}) = ||\mathbf{\Theta}||_1 = \sum_{l=1}^{L} ||\mathbf{W}^l||_1, \tag{14}$$

with the gradient for each layer l given as follows

$$\nabla_{\mathbf{W}}(\lambda \mathbf{\Omega}(\mathbf{\Theta})) = \lambda \cdot \text{sgn}(\mathbf{W}^l), \tag{15}$$

where sgn denotes the sign function. For the cost function with ℓ_1 regularisation, the expressions for the subgradients of each layer are thus

$$\nabla_{\mathbf{W}} C(\mathbf{\Theta}) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{a}_i^{l-1} (\boldsymbol{\delta}_i^l)^T + \lambda \cdot \text{sgn}(\mathbf{W}^l) \tag{16}$$

and

$$\nabla_{\mathbf{b}} C(\mathbf{\Theta}) = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{\delta}_i^l. \tag{17}$$

When applying an ℓ_2 norm regularisation, the additional term is given by

$$\mathbf{\Omega}(\mathbf{\Theta}) = ||\mathbf{\Theta}||_2^2 = \sum_{l=1}^{L} ||\mathbf{W}^l||_2^2, \tag{18}$$

with the subgradients of each layer following

$$\nabla_{\mathbf{W}}(\lambda \mathbf{\Omega}(\mathbf{\Theta})) = 2\lambda \mathbf{W}^l. \tag{19}$$

For the cost function with ℓ_2 regularisation, the expression for the subgradients is, as a consequence, written as

$$\nabla_{\mathbf{W}} C(\mathbf{\Theta}) = \frac{1}{n} \sum_{i=1}^{n} \mathbf{a}_i^{l-1} (\boldsymbol{\delta}_i^l)^T + 2\lambda \mathbf{W}^l \tag{20}$$

and

$$\nabla_{\mathbf{b}} C(\mathbf{\Theta}) = \frac{1}{n} \sum_{i=1}^{n} \boldsymbol{\delta}_i^l. \tag{21}$$

5. Binary and Multiclass Classification

For binary classification, the custom is to use the sigmoid activation function in the output layer, which is given by

$$\tilde{y}_i = \sigma(z_i^L) = \frac{1}{1 + e^{-z_i^L}}. \tag{22}$$

The binary cross-entropy (BCE) cost function may be written as

$$C_{\text{BCE}}(\mathbf{\Theta}) = -\frac{1}{n} \sum_{i=1}^{n} \Big[y_i \log(\tilde{y}_i) + (1 - y_i) \log(1 - \tilde{y}_i) \Big] + \lambda\, \Omega(\mathbf{\Theta}). \tag{23}$$

For the BCE, we use Equation 10 to find the expression for the outer error layer using

$$\boldsymbol{\delta}^L = (\mathbf{a}^{\mathbf{L}} - \mathbf{y}). \tag{24}$$

For multiclass classification, the standard practice is that the output layer uses the softmax activation function:

$$\tilde{\mathbf{y}}_i = \text{softmax}(\mathbf{z}_i^L). \tag{25}$$

The multiclass cross-entropy cost function is

$$C_{\text{softmax}}(\mathbf{\Theta}) = -\frac{1}{n} \sum_{i=1}^{n} \mathbf{y}_i^T \log(\tilde{\mathbf{y}}_i) + \lambda \Omega(\mathbf{\Theta}). \tag{26}$$

For multiclass cross-entropy, we find the outer layer error layer using Equation 10 and obtain the same result as Equation 24.

For the BCE and the multiclass cross-entropy the ℓ_1 and ℓ_2 norms result in the exact same expressions as obtained for the MSE in Equation 16 and Equation 17.

Furthermore, to classify both the binary and multiclass functions, we need to determine the accuracy of each training, which may be defined as

$$\text{Accuracy} = \frac{1}{n} \sum_{i=1}^{n} I(t_n = y_i). \tag{27}$$

Here, n is the number of targets, and I is the indicator function. I is set to 1 if the target t_n is equal to the predicted output y_i, otherwise it is set to 0.

B. Activation Functions

When we implement activation functions, we introduce non-linearity to our neural network. Similarly to the definition in Equation 22, the sigmoid function is given as

$$\sigma(z) = \frac{1}{1 + e^{-z}}, \tag{28}$$

and the derivative may be written as follows

$$\sigma'(z) = \sigma(z)[1 - \sigma(z)]. \tag{29}$$

The hyperbolic tangent function, tanh(z), may be defined as a variation of the sigmoid function following

$$\tanh(\text{z}) = \frac{e^z - e^{-z}}{e^z + e^{-z}}, \tag{30}$$

with its corresponding derivative given by

$$\tanh'(z) = 1 - \tanh^2(z). \tag{31}$$

The rectified linear unit, also named the ReLU function, is furthermore given as

$$\mathrm{ReLU}(z) = \max(0, z), \tag{32}$$

with its derivative

$$\mathrm{ReLU}'(z) = \begin{cases} 1, z > 0, \\ 0, z < 0. \end{cases} \tag{33}$$

The leaky ReLU activation function uses a fixed slope α (commonly set to $\alpha = 0.01$), and is further given by

$$\mathrm{LReLU}(z) = \begin{cases} z, z > 0, \\ \alpha z, z \leq 0, \end{cases} \tag{34}$$

with its corresponding derivative

$$\mathrm{LReLU}'(z) = \begin{cases} 1, z > 0, \\ \alpha, z \leq 0. \end{cases} \tag{35}$$

C. Cross-validation and the one-standard-error rule

The cross-validation curve near the minimum is flat, meaning that it is only possible to determine the minimum within the noise of the estimate [2]. This is the so-called *one-standard-error* rule. For cross-validation of the penalty parameter λ, if multiple values fall within this range, we use the following equation

$$\lambda_{1\mathrm{SE}} = \max\left\{\lambda : \mathrm{CV}_K\left(\lambda\right) \leq \mathrm{CV}_K\left(\lambda_{\min}\right) + \mathrm{SE}_K\left(\lambda_{\min}\right)\right\}, \tag{36}$$

stating that the most sensible choice is the most strongly regularised. Note that this is not a theorem, but rather a convention [2].

III. METHODS

A. Algorithms for network training

Some of the functions we have used for the analysis of the one-dimensional Runge problem and the classification of handwritten digits of the MNIST data set differ. The algorithms will therefore be presented as corresponding to two independent tasks. In the analysis of the one-dimensional Runge function, we use a set of training algorithms developed for this study, inspired by algorithms found in [6]. For the classification problem, we have borrowed and further developed functions from [6] making it possible for us to set up our FFNN classification network. Common to both set ups is the feed-forward pass.

1. *Feed-forward pass*

The feed-forward pass computes the output of the network for a given input. The input will be sent through the network one layer at a time, with each layer applying a linear transformation using its weights and biases, followed by a pre-defined activation function. The output of the final layer will correspond to the prediction of the network. The method is further illustrated in Algorithm 1.

Algorithm 1 Feed Forward Batch

Require:

X ▷ Input data, one sample per row
$L = (W, b)$ ▷ Layer composed of weights and biases
act ▷ Activation function for each layer

Do:

$a \leftarrow X$ ▷ Input to the first layer
for each layer $\ell = 1, \ldots, L$ **do**
  $z \leftarrow a\,W + b$ ▷ Linear transformation
  $a \leftarrow \mathrm{act}_\ell(z)$ ▷ Apply activation function
return a ▷ Output of the final layer

2. *Training algorithms for regression analysis*

For the regression analysis of the one-dimensional Runge problem, two different algorithms are used that share the same overall structure, however differ in how the parameters are updated. The network is trained over a set number of epochs, for every fiftieth epoch in addition to on the last one, the feed-forward pass computes the current predictions. The algorithms return the trained layers together with the MSE between the final predictions and the target values, which is the performance measure associated with our regression study.

In Algorithm 2, the fixed update rule employs a call to the optimiser step, which allows us to use the gradient descent methods from [5], such as *adam* and *rmsprop*. Each single weight matrix and bias vector has a corresponding individual optimiser state, which stores the information about the previous gradients and is then updated at every step. Here, the algorithm still uses the full data set in each update. For the regression problem, these gradients will be calculated using *JAX* [7]. In the algorithms, this is how we obtain dW_ℓ and db_ℓ. In principle, *JAX* works the same way as the backpropagation algorithm (which will be described in section III B), though the calculations are not as explicit. In section B in the appendix we have included a meta-analysis of these procedures.

Finally, Algorithm 3 extends the algorithm to work for stochastic gradient descent. Instead of computing the gradients on the complete data set, each epoch will be split into mini-batches of size M, with the parameters being updated once per mini-batch using gradients computed on that batch only. The learning rate may op-

Algorithm 2 Train Network Gradient

Require:

X ▷ Input data
$L = (W, b)$ ▷ Layer composed of weights and biases
act ▷ Activation function for each layer
Y ▷ Target values
$\gamma > 0$ ▷ Learning rate
E ▷ Number of epochs
$method$ ▷ Optimiser (plain, momentum, Adam, ...)
$state$ ▷ Optimiser state for each weight and bias
$pred$ ▷ predictions

Do:
for E **do**
  for each layer ℓ **do**
    $W_\ell, state_{W_\ell} \leftarrow \textsc{OptimiserStep}(W_\ell, dW_\ell, state_{W_\ell}, \gamma)$ ▷ Update weights
    $b_\ell, state_{b_\ell} \leftarrow \textsc{OptimiserStep}(b_\ell, db_\ell, state_{b_\ell}, \gamma)$ ▷ Update biases
  if $e \bmod 50 = 0$ **or** $e = E - 1$ **then**
    $pred \leftarrow \textsc{FeedForward}(X, layers, act)$ ▷ Current predictions
return $layers$, MSE$(pred, Y)$

tionally follow a schedule that changes its value during training.

Algorithm 3 Train Network Stochastic Gradient

Require:

X ▷ Input data
$L = (W, b)$ ▷ Layer composed of weights and biases
act ▷ Activation function for each layer
Y ▷ Target values
$\gamma > 0$ ▷ Learning rate
E ▷ Number of epochs
M ▷ Mini-batch size
$method$ ▷ Optimiser (plain, momentum, Adam, ...)
$state$ ▷ Optimiser state, updated by $\textsc{OptimiserStep}$
$pred$ ▷ predictions

Do:
for E **do**
  for each mini-batch B **do**
    Compute dW_ℓ, db_ℓ on B ▷ Mini-batch gradients
    $\gamma_t \leftarrow$ scheduled learning rate ▷ Optional schedule
    for each layer ℓ **do**
      $W_\ell \leftarrow \textsc{OptimiserStep}(W_\ell, dW_\ell, \gamma_t)$
      $b_\ell \leftarrow \textsc{OptimiserStep}(b_\ell, db_\ell, \gamma_t)$
  if $e \bmod 50 = 0$ **or** $e = E - 1$ **then**
    $pred \leftarrow \textsc{FeedForward}(X, layers, act)$
return $layers$, MSE$(pred, Y)$

B. Training algorithms for classification

For the classification network, two different algorithms are used from [6]: *Backpropagation* and *Train*. The *Backpropagation* algorithm computes how much each weight and bias will contribute to the error of the network. The algorithm first runs the data forward through the network, then estimates the difference between the predictions and the targets at the output layer and finally propagates this error backwards, layer by layer. At each layer, the gradients of the weights and biases are computed, to which an ℓ_1 or ℓ_2 regularisation term on the weights is added.

The *Train* algorithm employs the obtained gradients to train the network with stochastic gradient descent, in the same manner as Algorithm 3. For each epoch, the training data will be shuffled and split into small batches. After each batch the weights and biases are updated in the direction contribution to a reduced cost. The cost for the full training set is recorded after each epoch. The accuracy on the validation data set is then estimated to monitor the actual progress of the training.

IV. RESULTS AND DISCUSSION

A. Neural network for regression analysis

1. *First look at the model parameters*

To gauge the most sensible parameters to use for our model, we start by checking MSE values against a selection of learning rates for a given combination of number of hidden layers and nodes. To avoid information leakage from the test set, these values are computed using the training data only. Note that this is not a rigorous method for choosing parameters, but it serves as a sanity check and gives a reasonable initial estimate before we perform k-fold cross-validation. Throughout this analysis, unless stated otherwise, we use a number of 2000 data points, 100 epochs, a noise standard deviation $\sigma = 0.1$, and the output layer in the network is set up with no activation function. The data set is split in 80% training data, and 20% test data.

Figure 1 shows the obtained MSE values for the optimiser *adam* when using learning rates $\gamma \in [10^{-3}, 10^{-1}]$. The lowest acquired MSE value and corresponding learning rate are distinguished using dashed lines. Table I and Table II list the learning rates for the network with one hidden layer of 50 nodes and the network with two hidden layers of 100 nodes, respectively, together with the lowest MSE values obtained with *rmsprop*, *adam*, and plain gradient descent. We find that the prevailing smallest value for the MSE is found when using the *adam* optimiser method, for two hidden layers of 100 nodes. However, the difference in MSE between *adam* and *rmsprop* is so small, of the order of 10^{-5}, that it is difficult to make an informed choice between them. Since we have to make a choice, we will henceforth use the *adam* optimiser with a learning rate of $\gamma = 0.001$ when investigating the number of hidden layers and nodes in section IV A 2.

The predicted values for the Runge function based on the neural network set up with the *adam* optimiser

TABLE I. Lowest MSE values and corresponding learning rate γ, for a FFNN layout of one hidden layer of 50 nodes.

	plain	adam	rmsprop
MSE	0.0216	0.00994	0.00994
γ	0.0845	0.00774	0.00453

TABLE II. Lowest MSE values and corresponding learning rate γ, for a FFNN layout of two hidden layers of 100 nodes.

	plain	adam	rmsprop
MSE	0.0150	0.00990	0.00994
γ	0.0448	0.001	0.00453

are presented in Figure 2. We compare the results to the OLS results obtained in [5], where we found an MSE value of ≈ 0.0261 for a polynomial degree of 6. For the neural network, we obtain an MSE ≈ 0.0437 for the one-layer of 50 nodes case, and MSE ≈ 0.0315 for the two-layer of 100 nodes case. Hence, we see that the OLS generates a better model. From [5], we moreover obtained an MSE ≈ 0.0199 corresponding to the 25 k-fold cross-validation method, which likewise is found to be smaller than the MSE acquired from the neural network set up using the *adam* method. One possible explanation is overfitting; a neural network can easily fit the noise in the training data, which causes it to generalise poorly to unseen data. It may also indicate that the chosen learning rate is not optimal. However, the difference between the MSE values is small, and visually, the predictions from both network architectures provide a reasonable fit to the data.

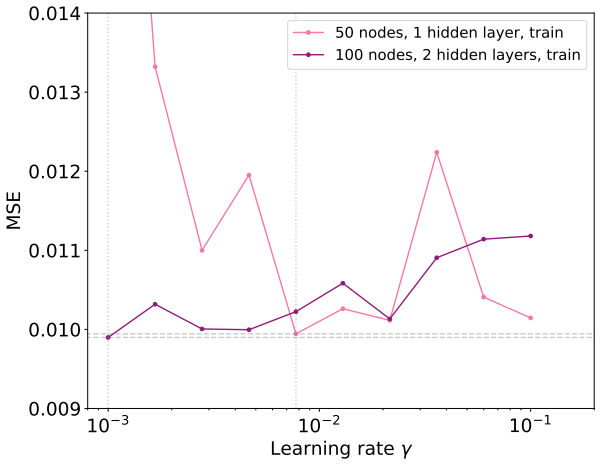


FIG. 1. MSE values for training data, using stochastic gradient descent with the optimiser method adam. The hidden layers used the sigmoid activation function.

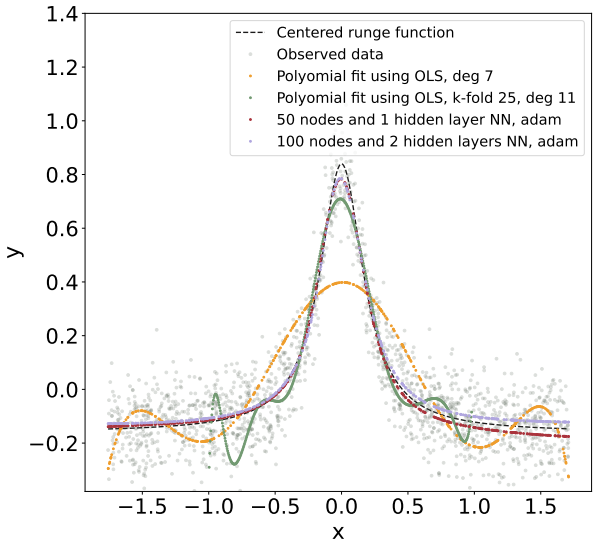


FIG. 2. Predicted values from neural network with different layouts compared to Runge's function, using stochasitc gradient descent with optimiser rmsprop, and learning rates $\gamma = 0.00774$ and $\gamma = 0.001$ for one hidden layer of of 50 nodes, and two hidden layers of 100 nodes respectively.

2. *Analysis of numbers of hidden layers and nodes*

We continue using the stochastic method *adam* with a learning rate $\gamma = 0.001$, as provided in section IV A 1. Although the number of epochs has not been explicitly investigated, we continue to use 100 epochs, as this gives satisfactory MSE values. We test the following combination of hidden layers $\in [1, 2, 3, 5, 10]$, and number of hidden nodes $\in [2, 5, 10, 20, 50, 100, 200]$. For the leaky ReLU activation function the results are presented in Figure 3. The results corresponding to ReLU and sigmoid may be found in Figure 13 and Figure 14 of section A, respectively.

Many of the different setup combinations reach a rounded MSE value of 0.010 for all activation functions, a value we are content with. Looking more closely at the decimals of each MSE value, it becomes clear that the lowest value is found when using the leaky ReLU activation function, reaching a value of ≈ 0.00975, for 5 hidden layers with 100 nodes in each layer. Given its strong performance, we retain the leaky ReLU activation function using 5 hidden layers with 100 nodes for the remaining experiments.

Having already decided which parameters we will continue using for our model we further analyse the general trends associated with changing the number of hidden layers and nodes. Figure 3 shows that some combinations yield distinctly higher MSE values than the general trend. This is especially the case for networks with two nodes per layer when the number of layers is at either

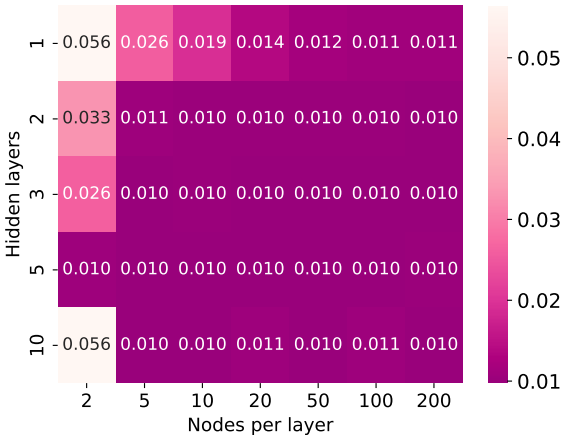


FIG. 3. MSE for a combination of number of hidden layers and nodes per layer, using the activation function leaky ReLU, a learning rate $\gamma = 0.001$, and the optimiser adam.

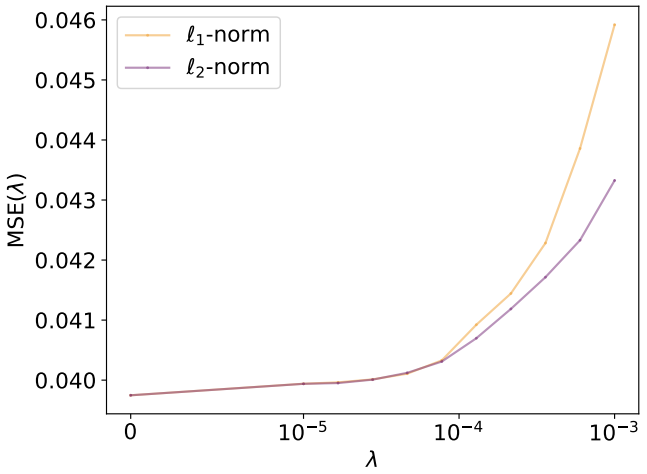


FIG. 4. MSE for $\lambda \in [10^{-5}, 10^{-3}]$ using $\gamma = 0.001$, epoch of 100, 5 layers and 100 nodes for the *Train Network Stochastic Gradient* algorithm with *adam* and by applying the leaky ReLU activation function.

the low or the high end of the chosen range. The combination of one hidden layer of two nodes, and ten hidden layers of two nodes generate the highest MSE values. A setup using two and three layers of 2 nodes also generates non-optimal MSE values. Furthermore, we see that using only one hidden layer can be a disadvantage if the number of nodes per layer is low; from 20 nodes and onward we however observe an improvement. It is not unexpected that a combination of a low number of hidden layers with a low number of nodes will in general be associated with higher MSE values. A one-hidden layer network may struggle to fit a function properly, still the universal approximation theorem says that *"A network with one hidden layer and a non-linear (non-polynomial) activation can approximate any continuous function on a bounded domain as accurately as we want, if the hidden layer is wide enough"* [8, Slide 14]. Our results may be explained by simply not having included enough nodes in the layer associated with these specific examples. The theorem also explains why, as the number of nodes per layer increases, a small number of hidden layers is sufficient to obtain satisfactory results: the network becomes wide enough to approximate the target function.

3. *Evaluating the effect of the ℓ_1 and ℓ_2 penalty terms*

Following the conclusions of the previous sections, we now analyse the effect of the regularisation parameter λ with ℓ_1 and ℓ_2 penalties. To this end, we train the network with Algorithm 3 using the Leaky ReLU activation function and *adam* optimiser, which gave the lowest MSE in section IV A 2. Based on the results in the same section, the network will be set up using 5 hidden layers with 100 nodes each. We use a learning rate of $\gamma = 0.001$, and $\lambda \in [10^{-5}, 10^{-3}]$.

TABLE III. MSE for Ridge and Lasso regression, using the optimal polynomial degree, corresponding penalty parameter λ, and 5-fold cross-validation, with noise standard deviation $\sigma = 0.1$. Values taken from [5].

	n	Degree	λ	MSE
Ridge	100	6	4.329×10^{-2}	0.0289
Lasso	250	10	3.594×10^{-6}	0.0157

The obtained results are shown in Figure 4. The lowest MSE values are obtained in the case where $\lambda = 0.0$. However, one may still compare the results obtained between the ℓ_1 and ℓ_2 norms for non-zero λ-values. Figure 4 illustrates that the ℓ_1-norm is associated with a higher MSE for $\lambda \in [10^{-5}, 10^{-3}]$ than the ℓ_2-norm. This statement has further been numerically verified, and the ℓ_2-norm will be used in the rest of the analysis.

To further assess the effect of the ℓ_1 and ℓ_2 penalties, we compare the MSE results in Table IV with the corresponding Lasso and Ridge regression results from [5], shown in Table III. For Ridge regression using the optimal polynomial degree of 6 with $\lambda = 4.329 \times 10^{-2}$ a MSE value of 0.0289 is achieved, see Table III [5]. The predictions from the neural network when using $\lambda = 4.329 \times 10^{-2}$ with an applied ℓ_2-norm reach an MSE

TABLE IV. MSE values obtained using Algorithm 3 with *Adam* and a Leaky ReLU activation function for 5 layers and 100 nodes using $\gamma = 0.001$ [5].

	λ	MSE
ℓ_2-norm	4.329×10^{-2}	0.0605
ℓ_1-norm	3.594×10^{-6}	0.0398

of 0.0605. Since the regression methods in [5] used a polynomial tuned specifically to reproduce the Runge function, while our neural network is a general-purpose approximator, it is not surprising that the Ridge regression results in Table III achieve a lower MSE.

Similarly, Lasso regression in Table III, with an optimal polynomial degree of 10 and $\lambda = 3.594 \times 10^{-6}$, obtains an MSE of 0.0157, whereas our neural network trained with ℓ_1 regularisation attains an MSE of 0.0398. Again, the regression methods from [5] outperform the neural network, further indicating that they are better suited to the one-dimensional Runge function.

Compared to standard regression methods, exemplified by the Lasso and Ridge results presented previously, neural networks have the advantage that features are learned in the hidden layers rather than being fixed beforehand as polynomial terms. Neural networks are seen as being more flexible; this flexibility, however, comes at a cost [2, Chapter 5]. The cost functions of Ridge and Lasso regression are convex, meaning the achieved minimum will be the global one. In contrast to neural networks, where the cost function is non-convex, and training is reliant on gradient descent and backpropagation, without any assurance that a global minimum will be found. The achieved results are dependent on a wide variety of hyperparameters; the learning rate, number of epochs, architecture, and activation function must, for instance, be defined. In the case of a relatively "simple" one-dimensional problem like the Runge function, where a well-tuned polynomial basis may be more than sufficient, the extra flexibility of a neural network will not pay off, consistent with the lower MSE obtained with Ridge and Lasso regression in Table III.

4. *Final k-fold cross validation and model selection*

Finally, we perform a k-fold cross-validation for our neural network. We therefore fix the network architecture (number of hidden layers and nodes per layer), the activation function, the number of epochs, and the regularisation norm at the values identified in the previous section, and restrict the cross-validation to the learning rate γ and the penalty parameter λ. However, we still carry out the cross-validation for plain gradient descent as well as for stochastic gradient descent with the *adam* and *rmsprop* optimisers.

Figure 5 illustrates the resulting MSE values obtained based on the training data for a combinations of γ and λ, using stochastic gradient descent with optimiser *rmsprop*. The non-annotated cells represent MSE values that are larger than 0.1, a limit we have set in order to make the features of the heatmap readable. We justify this choice by noting that any model with an MSE above 0.1 performs too poorly to be of practical interest, and would not be a model we could recommend. Furthermore, the annotated cells show all reached MSE values below 0.1. However, notice that the one-standard-error threshold previously explained in section II C has not yet been applied, meaning all MSE values have been included in Figure 5. After having applied the one-standard-error threshold, we identify a $\lambda = 0.0005$ as the largest penalty parameter, following the convention stated in section II C, with a corresponding $\gamma = 0.001$, as displayed in Table V. The table further illustrates the same procedure when using the *adam* optimiser, and plain gradient descent. *rmsprop* yields the lowest validation MSE (≈ 0.0100), and we will therefore adopt this optimiser in our final model. In section IV A 1, when varying a single parameter at a time we concluded on using *adam* in our recommended model; it is nevertheless not unexpected that after performing a more thorough analysis we end up with a different result.

Our final recommended model uses five hidden layers with 100 nodes each, the Leaky ReLU activation function, and ℓ_2 regularisation. It is trained for 100 epochs with the *rmsprop* optimiser, using a learning rate of $\gamma = 0.001$ and a penalty parameter of $\lambda = 0.0005$. Training the neural network with these hyperparameters gives the final fit shown in Figure 6, with a test MSE ≈ 0.0272. We consider this a satisfactory result that supports our choice of hyperparameters. Although optimising the hyperparameters reduced the test MSE from ≈ 0.0315 (section IV A 1) to ≈ 0.0272, our neural network is still outperformed by Ridge regression from [5], which achieved an MSE ≈ 0.0155.

This is not entirely unexpected, and several of the factors discussed earlier help explain it. First, the Runge function is a smooth, one-dimensional problem for which a well-tuned polynomial basis is already sufficient, so the additional flexibility of the neural network offers little benefit. Second, whereas the convex cost function of Ridge regression guarantees that the global minimum is found, the cost function of the neural network is non-convex, and gradient-based training may therefore converge to a local minimum. Third, the performance of the network depends on many hyperparameters. Since we only cross-validated γ and λ, while keeping the architecture, activation function, and number of epochs fixed, a better-performing configuration may still exist. Finally, the flexibility of the network makes it prone to overfitting, as it can easily adapt to random noise in the training data.

We also compare the fitted model with a baseline network without hidden layers, shown together with the final fit in Figure 6. An FFNN without hidden layers cannot approximate a general nonlinear continuous function [9]. The baseline model's failure to reconstruct the Runge function in Figure 6 is therefore consistent with the cited literature. In the case of nonlinearity, the introduction of activation functions is necessary. The baseline model reaches a test MSE of ≈ 0.1025, almost four times larger than the MSE of ≈ 0.0272 obtained with our final network. This shows that the hidden layers add significant value to the network. As the baseline differs from our model only in the absence of hidden layers, any improvement in performance can be attributed to them directly.

TABLE V. Learning rate γ, penaty parameter λ, and resulting MSE for optimisers plain, adam, and rmsprop, for the ℓ_2 norm.

ℓ_2	gamma	lambda	MSE
plain	0.1	0.005	0.0533
adam	0.001	0.001	0.0349
rmsprop	0.001	0.0005	0.0272

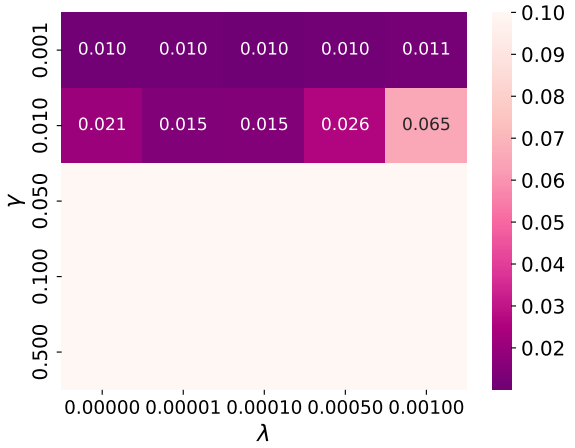


FIG. 5. MSE for cross-validation of parameters γ and λ for stochastic gradient descent with optimiser rmsprop, using 5 hidden layers of 100 nodes, and the ℓ_2 norm.

To summarise, the FFNN trained with the optimal hyperparameters found in this section achieves a sufficiently low MSE for us to recommend the model. It does not, however, outperform the Ridge regression from [5]. This may reflect known limitations of neural networks, such as their tendency to overfit noise. Rather than arguing against neural networks, this highlights the importance of choosing a method suited to the design of the chosen problem [2]. For the Runge function, Ridge regression appears to be the more suitable approach, although further analysis, as outlined in section V A, would be needed to state this with confidence.

B. Neural network adapted to a classification problem

The following analysis is based on the MNIST dataset of handwritten digits [4], with the aim of selecting a model that classifies the digits accurately. As in our study of neural networks for the one-dimensional Runge function regression problem, we examine a range of hyperparameter choices, including the number of hidden layers and nodes, the activation function, the learning rate, the type of regularisation norm, and the penalty parameter. The number of epochs is fixed at 50 throughout the analysis. Although this is also a hyperparameter that could be optimised, we found 50 epochs to be sufficient for the models' accuracy to converge, while keeping the computational cost manageable.

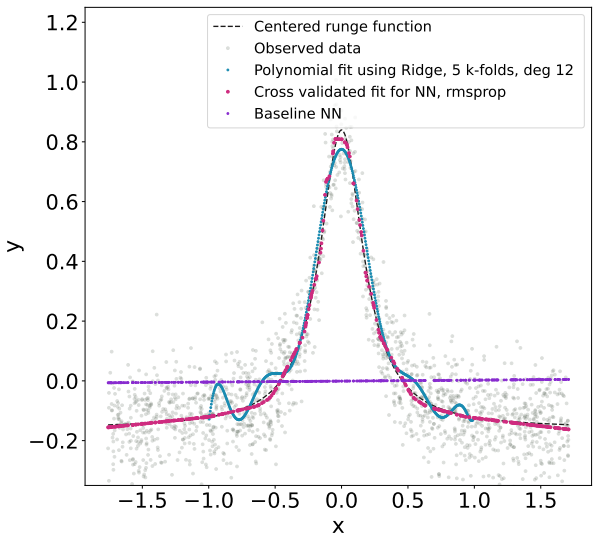


FIG. 6. Test predictions from the final neural network, trained with stochastic gradient descent using the *rmsprop* optimiser with a Leaky ReLU activation function, $\gamma = 0.001$, $\lambda = 0.0005$.

1. *Selection of the best architecture*

To select the architecture, we first perform a grid search with k-fold cross-validation over different learning rates and hidden layer configurations. We then keep the combinations whose mean accuracy was within one standard deviation, computed across the k folds, of the best-performing model. The obtained results are listed in Table VI. The best results are obtained with two hidden layers of 100 and 50 nodes, although the accuracy varies with the learning rate.

TABLE VI. Cross-validated accuracy for two different architectures and learning rates $\gamma \in [0.01, 0.6]$ within one standard deviation of the best model. All models use the ReLU activation function and the cross-entropy cost function, and are trained with plain gradient descent on a reduced training set of 10 000 samples.

Architecture	γ	Accuracy (%)
$[784, 100, 50, 10]$	0.3368	**95.31**
$[784, 100, 50, 10]$	0.5000	95.27

In the setup found in Table VI, the ReLU activation function was used, helping minimise the computational

TABLE VII. Mean execution time and standard deviation for the activation functions sigmoid, tanh, ReLU, and Leaky ReLU, using the optimal values for the penalty parameter $\lambda = 0$ and learning rate $\gamma = 0.5186$. Cross-entropy cost function with two hidden layers of 100 and 50 nodes.

Activation Function	Mean Time (s)	SD (s)
Sigmoid	70.00	12.85
Tanh	81.20	18.93
ReLU	**54.40**	**9.95**
Leaky ReLU	57.80	12.16

time needed. Based on the results presented in Table VII, ReLU outperformed the other three activation functions sigmoid, tanh and Leaky ReLU by computational time, as seen in the mean and the standard deviation, after running the four activation functions 5 times. The exact time it takes to train the model depends on different factors, for instance, the specifications of the computer running the algorithm. Variations within the same computer may also arise.

This is clearly seen in the standard deviation in Table VII, with the tanh activation function having a mean time of 81.20 seconds, but with a standard deviation of 18.93 seconds. This test was done to see which activation function should be used as a default in the analysis prior to selecting a final activation function. Fixing the activation function to ReLU during this search may bias the selected parameters towards ReLU, but repeating the grid search for every activation function would be computationally expensive. For the same reason, the cross-validation was performed on a reduced training set. The reduced set consists of 10 000 digits taken from the full training set of 60 000, selected such that the overall distribution of the classes (the digits from 0–9) stays the same as the full training set. This training set is selected once, and will not be resampled.

Nevertheless, the two-layer architecture with 100 and 50 nodes clearly performed best, outperforming all other architectures by more than one standard deviation as shown in Table VI. The only competing models within this range use the above-mentioned architecture with learning rates $\gamma = 0.3368$ and $\gamma = 0.5000$.

2. *Selection of Learning Rate and Norm*

As presented in the method for architecture selection, we perform a grid search with cross-validation to find the best combination of penalty parameter and learning rate. All pairs within one standard deviation of the highest accuracy are listed in Table XII in section A 2 in the appendix, with the best pair being $\gamma = 0.3050$ and $\lambda = 0.0005$. The cross-validation uses the reduced training set, the ReLU activation function, and plain gradient descent. The selected pairs are then retrained on the full training set, giving the results in Table VIII. Here, the two models without regularisation ($\lambda = 0$) and learning rates $\gamma = 0.4525$ and $\gamma = 0.6000$ outperform all others, both with an accuracy of $> 98\%$.

TABLE VIII. Estimated accuracy for learning rates $\gamma \in [0.01, 0.6]$ and $\lambda \in [0, 0.0001]$ using the full training set. Cross-entropy cost function and ReLU activation function with two hidden layers of 100 and 50 nodes, using plain gradient descent and ℓ_2 regularisation.

γ	λ	Accuracy (%)
0.3050	0.00025	96.26
0.3050	0.00050	95.01
0.4525	0.00000	98.10
0.4525	0.00025	96.02
0.4525	0.00050	93.11
0.6000	0.00000	**98.16**
0.6000	0.00025	94.87

The ranges for the learning rate and penalty parameter in Table VIII were first chosen broadly to identify patterns, and then narrowed to the region with the best results. Comparing the results in this table with the intermediate results from the smaller test set, shown in Table XII in section A 2 in the appendix, reveals two observations. First, it seems that larger training sets are associated with an overall accuracy increase for the results with little to no regularisation. For example, for the smaller set, the trained model with learning rate $\gamma = 0.4525$ and penalty parameter $\lambda = 0$ obtains an accuracy of 95.05%, whilst with the full training set, the accuracy becomes 98.10%. The obtained accuracies suggest that including a larger training set will help achieve more accurate models. The quality of the data will also factor into the obtained accuracy.

Secondly, with the reduced training set, adding a penalty term consistently improved accuracy, whereas for the full training set, the unregularised models performed best. An example of this for the reduced training set is at a learning rate of $\gamma = 0.4525$, where penalty parameters of $\lambda = 0.00025$ and $\lambda = 0.00050$ achieved accuracies of 95.24% and 95.13% respectively, compared with 95.05% for $\lambda = 0$. The pattern is however not transferable for the larger training set where $\lambda = 0$ yields 98.10%, $\lambda = 0.00025$ decreases the reached accuracy to 96.02% and a $\lambda = 0.00050$ obtains 93.11%. The described phenomena may be explained using different factors; for instance, a larger training set may reduce the risk of overfitting and thereby the need for explicit regularisation. The relatively low number of epochs (50 epochs) limits the model's opportunity to overfit, meaning the penalty mainly restricts the optimisation of the cost function and slows down training. The reasoning is expected to apply to ℓ_1 regularisation as well, and will as a consequence not be analysed separately. When training on the full train-

ing set, no regularisation was applied, as this yielded the highest accuracy in Table VIII.

To refine the choice of learning rate, we additionally run cross-validation on the reduced training set with a larger grid of values for γ . The learning rates within one standard deviation of the most accurate model are listed in Table XIII in section A in the appendix. These learning rates are relatively large, and choosing the largest might seem appealing, as it would correspond to the fastest convergence. However, a large learning rate also risks unstable training, so we perform further testing to analyse how the learning rates behave on the full training set.

TABLE IX. Cross-validated accuracy for learning rates γ within one standard deviation of the best model. All models use the ReLU activation function and the cross-entropy cost function, and are trained with plain gradient descent on the full training set of the MNIST.

γ	Accuracy (%)
0.3966	98.08
0.5186	**98.29**
0.5797	98.26

Based on the additional conducted analysis in Table IX, we select $\gamma = 0.5186$ as our final learning rate for our recommended model, used with plain gradient descent.

3. *Selection of Activation Function*

This part of the analysis determines the best-performing activation function among the four introduced in section II: sigmoid, tanh, ReLU and Leaky ReLU. The activation function is selected differently from the previously determined hyperparameters. Instead of cross-validation, we train the model with the optimal parameters from the previous sections (two hidden layers with 50 and 100 nodes respectively, learning rate $\gamma = 0.5186$ and $\lambda = 0$) for each activation function, repeating the training with five different seeds (see Table X for the exact seeds). The mean accuracy and standard deviation for each activation function are listed in Table X and visualised in Figure 7. ReLU achieves the highest mean accuracy, 98.16%, with the lowest standard deviation, 0.03633%. Leaky ReLU also performs well with an accuracy of 98.14% staying within the standard deviation of ReLU, its own standard deviation is however considerably higher reaching a value of 0.06462%. The sigmoid and tanh activation functions perform noticeably worse, with more than 6 in standard deviation separating them from ReLU (SD= 0.03633%).

Since the ReLU activation function was the basis for our selection of the optimal architecture, penalty parameter, and learning rate, a bias might have been introduced towards favouring ReLU over the other activation functions. Repeating the full hyperparameter search for each activation function would address the issue, and could be explored in future work. Based on the present analysis, we select ReLU as the optimal activation function for the dataset.

TABLE X. Mean accuracy and standard deviation of activation functions after running 5 separate seeds (2026, 2027, 2028, 2029 and 2030) with 50 epochs for the optimal penalty parameter $\lambda = 0$ and learning rate $\gamma = 0.5186$ with the cross-entropy cost function, two hidden layers with 100 and 50 nodes.

Activation Function	Mean Accuracy (%)	SD (%)
Sigmoid	97.89	0.07248
Tanh	97.93	0.06493
ReLU	**98.16**	**0.03633**
Leaky ReLU	98.14	0.06462

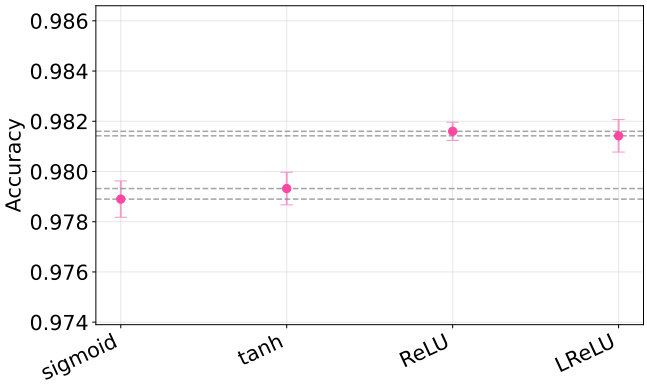


FIG. 7. Accuracy of activation functions after running 5 separate seeds (2026, 2027, 2028, 2029 and 2030) with 50 epochs for the optimal penalty parameter $\lambda = 0$ and learning rate $\gamma = 0.5186$ with the cross cost function, two hidden layers with 100 and 50 nodes.

4. *Final Model Selection*

The final model uses the optimal hyperparameters found in this analysis: two hidden layers with 100 and 50 nodes, the ReLU activation function, a learning rate of $\gamma = 0.5186$, and no regularisation. It is trained with plain gradient descent for 50 epochs.

The model is trained on the full training set. A logistic model (without hidden layers) is additionally trained on the same full dataset and will be compared to our developed neural network. We moreover use the Scikit-learn python library [10] as a second comparison for our model. The obtained results are presented in Table XI, and illustrate that our model outperforms both the logistic model and the Scikit-learn model by $\sim 5.8\%$. The comparison was performed using a test set of 10000, meaning that our

TABLE XI. Accuracy comparison of the baseline models and the final selected model.

Model	Accuracy (%)
Scikit-learn Model	92.47
Baseline Logistic Regression	92.37
Final Selected Model	**98.26**

model manages to classify about 580 more digits than the logistic models.

The confusion matrix in Figure 8 shows how often digits get confused; the diagonal elements correspond to correct predictions. The most common error for our model is confusing 4s and 9s: eleven images of 4s were classified as 9s, and seven images of 9s were classified as 4s. The model also tends to confuse 5s with 3s and 7s with 2s.

V. CONCLUSIONS

In this report, we build a feed-forward neural network (FFNN). Training was performed by combining a feed-forward algorithm setup with backpropagation, first by explicitly calculating the gradients of both the weights and biases, and secondly by employing *JAX*. Our analysis researches the use of FFNNs in two different problems: the regression exercise associated with the one-dimensional Runge function and the classification of handwritten digits from the MNIST dataset.

For the regression analysis, our starting point is the architectures of one and two hidden layers, respectively carrying 50 and 100 nodes. The first step of our analysis investigates a small selection of parameters at a time. By comparing MSE values for the training data, we find that the model setup with two hidden layers and 100 nodes performs better, achieving an MSE ≈ 0.00990 when using a learning rate $\gamma = 0.001$, and stochastic gradient descent with the *adam* optimiser. The optimiser *rmsprop*, as well as plain gradient descent, were moreover investigated; however, resulting in higher MSEs.

The results associated with other combinations of hidden layers and nodes were further investigated; the optimal combination was found to be five hidden layers of 100 nodes. The lowest MSE value ≈ 0.00975 was reached when employing the Leaky ReLU activation function. We further examined the impact of adding a penalty term to the cost function; our intermediate analysis found that, when regularisation was included, the ℓ_2 norm generally performed better than the alternative. However, the lowest MSE values were obtained without regularisation. When investigating the non-zero penalty parameter model, the MSE results were compared to our results obtained using the optimal λ-value from [5] for the Ridge and Lasso analysis. We therefore conclude that, when regularisation is applied, the ℓ_2 norm is the better choice for our model.

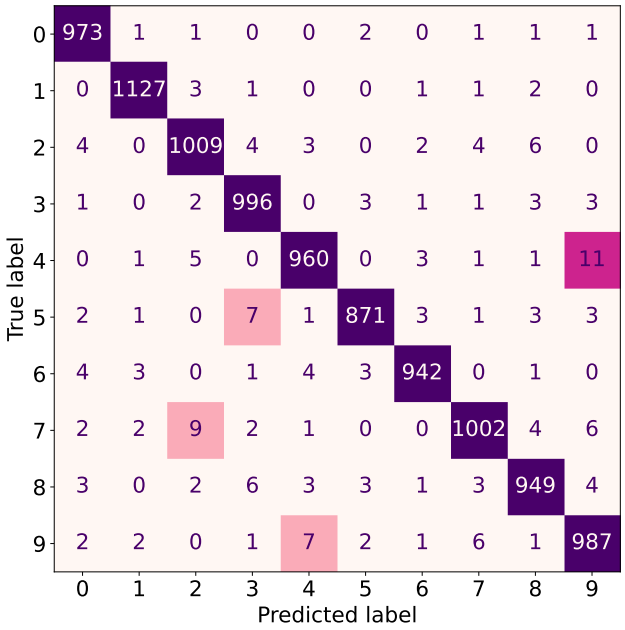


FIG. 8. Confusion matrix built by the optimal classification model with the cross-entropy cost function, 50 epochs, two hidden layers with 100 and 50 nodes respectively, learning rate $\gamma = 0.51862$ with plain gradient descent, no norm $\lambda = 0$ and ReLU activation function.

Finally, for the regression analysis, we use cross-validation to compare plain gradient descent (MSE ≈ 0.0533), and stochastic gradient descent with *rmsprop* (MSE ≈ 0.0272) and *adam* (MSE ≈ 0.0349) using the ℓ_2 optimal norm. Contrary to our earlier findings, where *adam* reached the lowest MSE, the best results are obtained when operating *rmsprop*. To summarise, our final model consists of 5 hidden layers with 100 nodes each, using the Leaky ReLU activation function. It is trained for 100 epochs with the *rmsprop* optimiser at a learning rate of $\gamma = 0.001$, and regularised using the ℓ_2 norm with a penalty parameter of $\lambda = 0.0005$.

For the classification task, we extend our k-fold cross-validation to also determine the optimal architecture, norm, and activation function. The optimal architecture, consisting of two hidden layers with 100 and 50 nodes respectively, achieves an accuracy of 95.31% when combined with a learning rate of $\gamma \approx 0.3367$ and the ReLU activation function. An additional grid search is performed, finding the optimal learning rate and norm. The best results are obtained when excluding regularisation using a learning rate $\gamma = 0.5186$; an accuracy of 98.29% is achieved.

To select the activation function, we no longer utilise cross-validation; instead, the previously found parameters are kept fixed and the sigmoid, tanh, ReLU, and Leaky ReLU activation functions are tested across five selected different seeds. ReLU generates the highest ac-

curacy, presenting a value of 98.16%. Consequently, the final classification model comprises two hidden layers of 100 and 50 nodes, respectively, employing the ReLU activation function. The network is optimised using plain gradient descent with a learning rate of $\gamma = 0.5186$ without regularisation. Our FFNN is subsequently compared with two logistic regression models: the Scikit-Learn implementation, as well as a baseline logistic regression of our own optimised parameters. Our recommended FFNN outperforms both, achieving an accuracy of 98.26%, compared with 92.47% and 92.37% for the Scikit-Learn and in-house logistic regression models, respectively. The classification analysis supports the notion that neural networks benefit from large amounts of training data, as the accuracy tends to increase with the number of training samples.

A. Topics for future work

A natural extension of this work would be a more extensive cross-validation for the regression analysis, in which additional hyperparameters are varied, such as the number of hidden layers and nodes, the number of epochs, and the activation function. For the regression task, it would also be of interest to investigate how the neural network responds to varying the number of data points and applying different levels of noise to the data. For the classification task, future work could explore whether data quality can be defined and measured quantitatively, and how it affects the overall accuracy of the model.

[1] K. Miller, "From Brain to Machine: The Unexpected Journey of Neural Networks | Stanford HAI."

[2] M. Hjort-Jensen, "Machine learning and the physical sciences: From discriminative methods to generative methods and reinforcement learning." https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/BookML/Lecturebook.pdf. MachineLearningUiO, accessed 22 September 2026.

[3] C. Runge, "Über empirische funktionen und die interpolation zwischen äquidistanten ordinaten," *Zeit. Math. Phys.*, vol. 46, pp. 224–243, 1901.

[4] Y. LeCun, C. Cortes, and C. Burges, "Mnist handwritten digit database." http://yann.lecun.com/exdb/mnist, 2010.

[5] M. O. Pilskog, S. S. Jorgensen, and M. M. Mumtaz, "An in-depth analysis of key ml concepts in light of the runge function," *FYS-STK4155, Project 1*, 2026.

[6] M. Hjort-Jensen, "Machine learning course materials." GitHub. https://github.com/EducationalMaterialUiO/MachineLearningUiO/tree/main/doc. Accessed 4 October 2026.

[7] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018.

[8] M. Hjort-Jensen, "Week 42 monday." GitHub. https://github.com/EducationalMaterialUiO/MachineLearningUiO/blob/main/doc/WeeklyMaterial/week42/week42monday.pdf. Accessed 2 November 2026.

[9] F. Scarselli and A. Chung Tsoi, "Universal Approximation Using Feedforward Neural Networks: A Survey of Some Existing Methods, and Some New Results," *Neural Networks*, vol. 11, pp. 15–37, Jan. 1998.

[10] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[11] The JAX Authors, "JAX: High performance array computing." https://docs.jax.dev/en/latest/, 2024. Accessed: November 5, 2026.

Appendix A: Additional analysis material

1. Additions: Neural network for regression

In this section, we present the results from models which have been outperformed by our recommended model. The methods for obtaining these results closely follow the procedures introduced in section IV A. As for our main analysis, once we have deduced an optimal hyperparameter, we keep it henceforth in our analysis, and employ the value as fixed in the final k-fold cross-validation analysis. The prior statement excludes γ and λ, as these are the hyperparameters we cross-validate.

a. *First look at the model parameters*

Figure 9 and Figure 10 show the MSE values obtained for the training data for learning rates $\gamma \in \left[10^{-3}, 3 \times 10^{-1}\right]$, when using plain gradient descent and stochastic gradient descent with the *rmsprop* optimiser, respectively.

In Figure 11 and Figure 12, we present the fit for the first look at the model when using the lowest learning rates found in Figure 9 ($\gamma = 0.0845$ with corresponding MSE ≈ 0.216 for one hidden layer of 50 nodes, and $\gamma = 0.0448$ with corresponding MSE ≈ 0.0150 for two hidden layers of 100 nodes) and Figure 10 ($\gamma = 0.00453$ with corresponding MSE ≈ 0.00994 for both architectures of one hidden layer of 50 nodes, and two hidden layers of 100 nodes), further stated in Table I and Table II.

It is clear that employing plain gradient descent with $\gamma = 0.0845$ and $\gamma = 0.0448$ for one layer of 50 hidden nodes and two layers of 100 hidden nodes, respectively, as displayed in Figure 11, does not provide satisfactory results. Our neural network struggles to recreate the distinct shape of the Runge function for both presented layouts. This can be mitigated by substantially increasing the number of epochs.

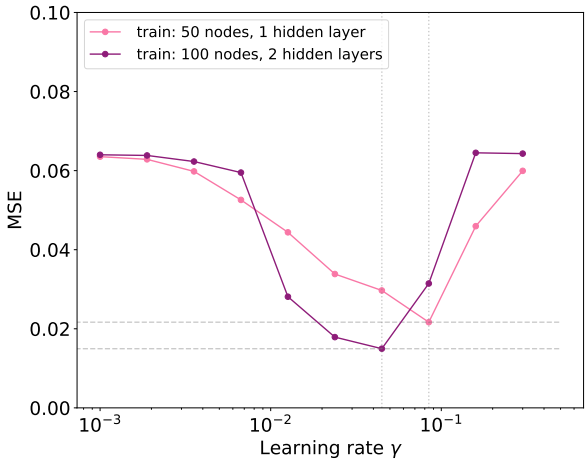


FIG. 9. MSE values for training data, using plain gradient descent. The hidden layers used the sigmoid activation function.

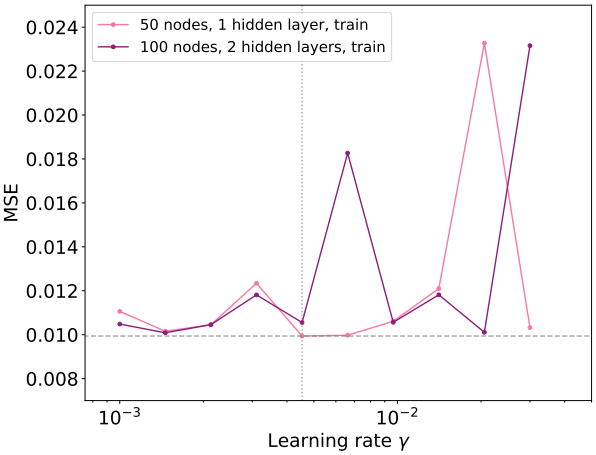


FIG. 10. MSE values for training data, using stochastic gradient descent, with optimiser rmsprop. The hidden layers used the sigmoid activation function.

b. *Analysis of numbers of hidden layers and nodes*

The resulting MSE values for the combinations of hidden layers $\in [1, 2, 3, 5, 10]$, and number of hidden nodes $\in [2, 5, 10, 20, 50, 100, 200]$ for the activation functions sigmoid and ReLU are presented in Figure 13, Figure 14 respectively. For activation function ReLU, we identify the optimal architecture with ten hidden layers of ten nodes, resulting in an MSE ≈ 0.00975. For activation function sigmoid, we identify an optimal architecture with two hidden layers of 200 nodes, which results in an MSE ≈ 0.00989. For this MSE, including a higher number of significant digits reveal that this MSE is lower than for *adam*, so this is the optimiser we chose to continue utilising in our analysis.

c. *Final k-fold validation and model selection*

Figure 15 and Figure 16 show the MSE for the cross-validation of γ and λ, for plain gradient descent and stochastic gradient descent respectively. Using plain gradient decent the optimal combination of the learning rate and penalty parameter are found to be $\gamma = 0.1$ and $\lambda = 0.005$, with an MSE ≈ 0.0185. For the optimser *adam*, the optimal values are $\lambda = 0.001$ and $\lambda = 0.001$, resulting in an MSE ≈ 0.0101.

The final fit for the model using plain, and stochastic gradient descent for optimiser *adam*, is shown in Figure 17, with a final MSE of ≈ 0.0489, and Figure 18 with a final MSE value of ≈ 0.0370, respectively.

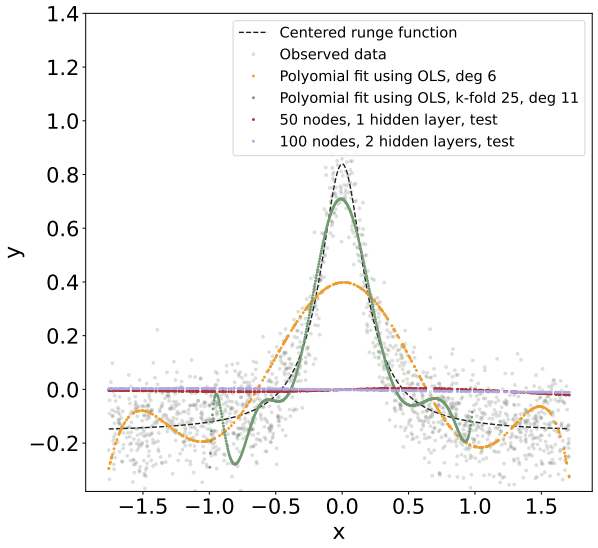


FIG. 11. Predicted values from neural network with different layouts compared to Runge's function, using plain gradient descent, and learning rates $\gamma = 0.0845$ and $\gamma = 0.0448$ for one hidden layer of of 50 nodes, and two hidden layers of 100 nodes respectively.

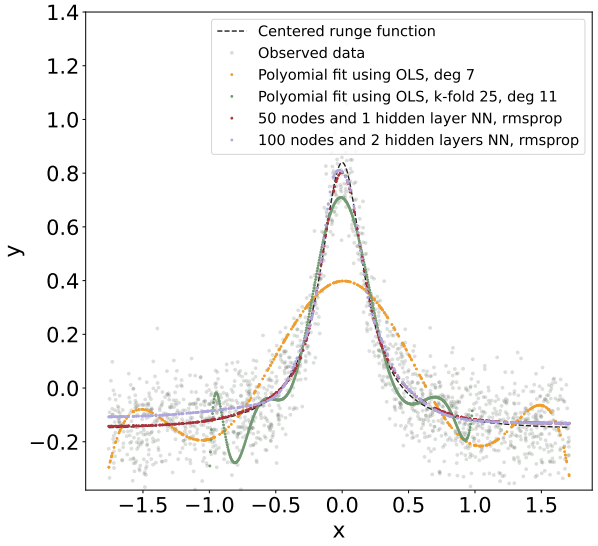


FIG. 12. Predicted values from neural network with different layouts compared to Runge's function, using stochasitc gradient descent with optimiser rmsprop, and learning rate $\gamma = 0.00453$ for both architectures of one hidden layer of 50 nodes, and two hidden layers of 100 nodes.

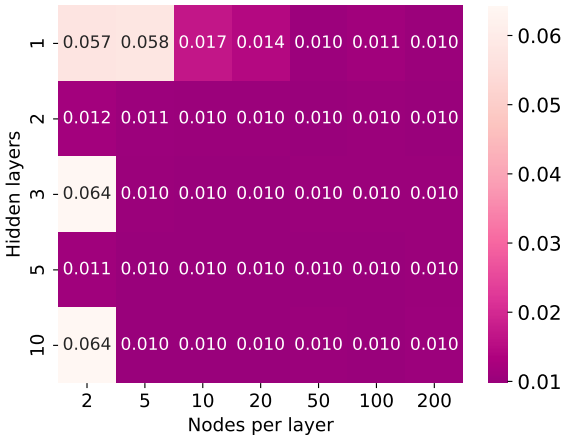


FIG. 13. MSE for a combination of number of hidden layers and nodes per layer, using the activation function ReLU, a learning rate $\gamma = 0.001$, and the optimiser adam.

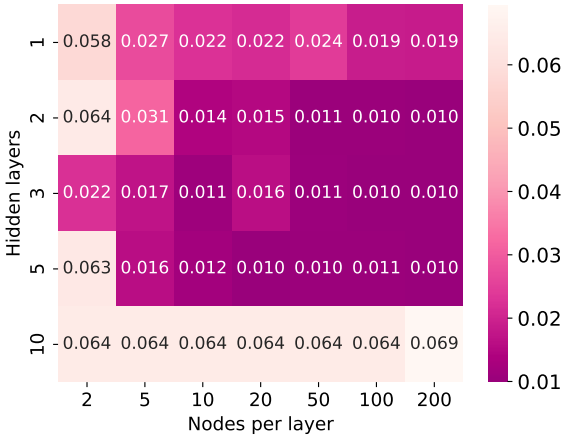


FIG. 14. MSE for a combination of number of hidden layers and nodes per layer, using the activation function sigmoid, a learning rate $\gamma = 0.001$, and the optimiser adam.

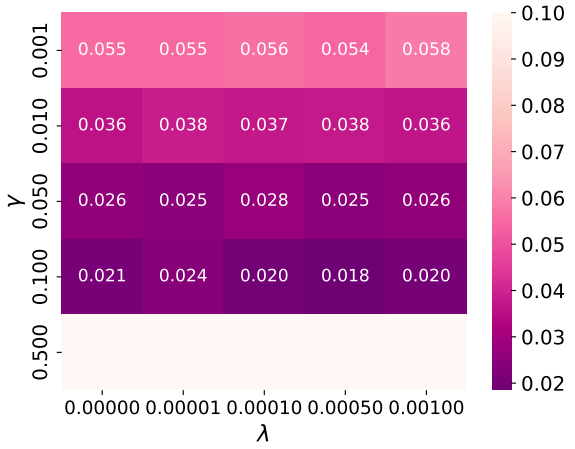


FIG. 15. MSE for cross-validation of parameters γ and λ for plain gradient descent, using 5 hidden layers of 100 nodes, and the ℓ_2 norm.

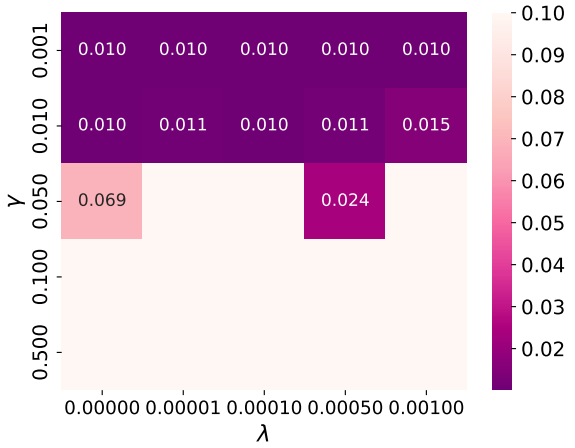


FIG. 16. MSE for cross-validation of parameters γ and λ for stochastic gradient descent with optimiser adam, using 5 hidden layers of 100 nodes, and the ℓ_2 norm.

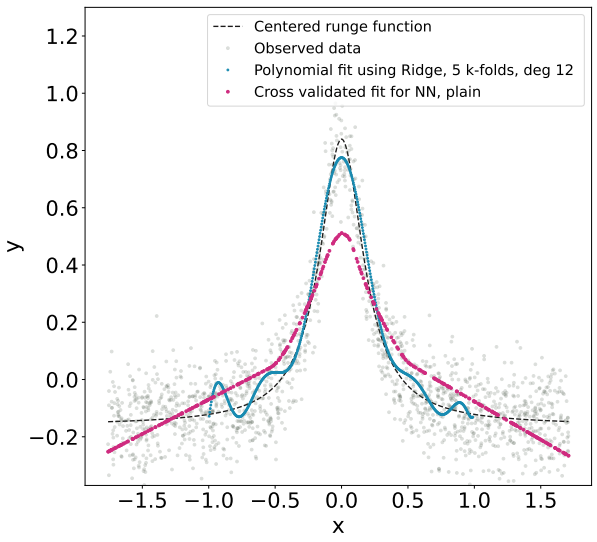


FIG. 17. The test predictions from our neural network with the parameters given in section IV A 4, using plain gradient descent.

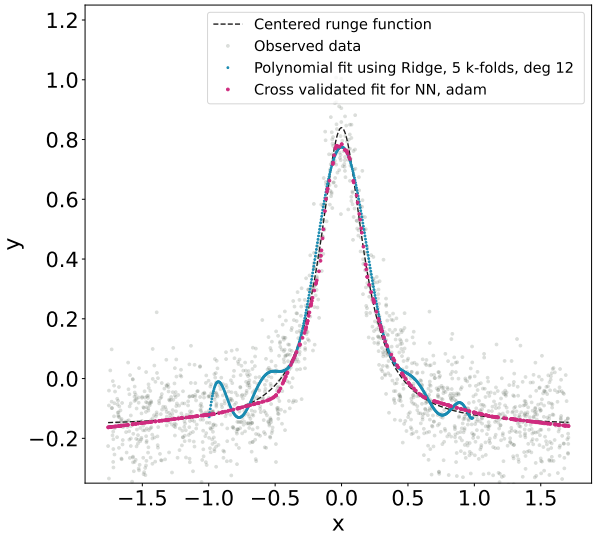


FIG. 18. The test predictions from our neural network with the parameters given in section IV A 4, using stochastic gradient descent with optimiser adam.

2. Additions: Neural network for classification

a. *Finding eligible values for model parameters*

This section presents the intermediary values obtained when training the model on a smaller training set of 10 000 digits instead of the full MNIST training set, finding eligible values within one standard deviation of the optimal parameter.

TABLE XII. Accuracy for different values of γ and λ using a smaller training set of 10 000 digits from the MNIST dataset and cross-validation. Cross-entropy cost function and ReLU activation function with two hidden layers of 100 and 50 nodes, using plain gradient descent and ℓ_2 regularization.

γ	λ	Accuracy (%)
0.3050	0.00025	95.12
0.3050	0.00050	**95.32**
0.4525	0.00000	95.05
0.4525	0.00025	95.24
0.4525	0.00050	95.13
0.6000	0.00000	95.23
0.6000	0.00025	95.21

Table XII presents the values for the penalty parameter and learning rate selected to be further analysed in the report.

TABLE XIII. Accuracy for different values of γ using a smaller training set of 10 000 digits from the MNIST data set and cross-validation. Cross-entropy cost function and ReLU activation function with two hidden layers of 100 and 50 nodes, using plain gradient descent.

γ	Accuracy (%)
0.3966	95.26
0.5186	95.24
0.5797	**95.34**

Table XIII reveals the values for the learning rate selected to be further analysed in the report.

b. *Visualisation of the final model*

A visualisation of the final model for the classification of the MNIST digits selected in the report will be presented hereafter.

Figure 19 shows some of the predictions for our recommended model for the classification exercise. The model assigns a certain probability that the image is within a distinct class of the model, and the highest probability becomes the selected prediction. In this case, all five images are correctly predicted.

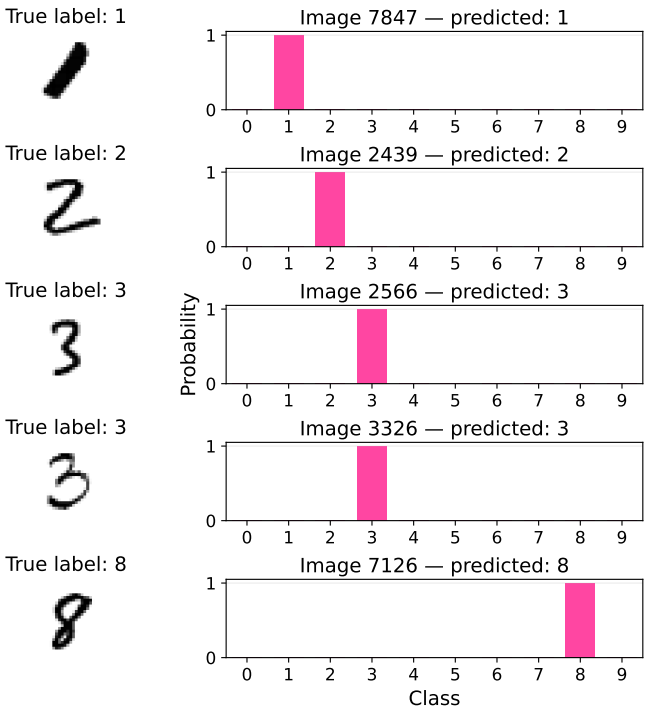


FIG. 19. Five selected images from the test set predicted by the optimal model with the cross-entropy cost function, 50 epochs, two hidden layers with 100 and 50 nodes respectively, learning rate $\gamma = 0.5186$ with plain gradient descent, no norm $\lambda = 0$ and ReLU activation function.

Appendix B: Meta-analysis on use of JAX automatic differentiation versus backpropagation algorithm

There is one divisive concern in the implementation of neural networks for both the regression and classification exercises. In the regression case, an algorithm employing cost functions defined through *JAX* is utilised and automatically differentiated using its built-in `grad()` function. The functionality is further described in *JAX*-documentation [11]. In simple terms, *JAX* finds the subgradients of the different layers. The analysis in this report assumes that this procedure is identical to the backpropagation algorithm described in section III, which is used in the classification case. The extended analysis presented in this section aims to illustrate the equivalence of the two described methods, further proving that the preference of one over the other shall only be based on computational cost and optimisation. The code used for this meta-analysis may be found in `meta_analysis.py` in our GitHub repository[2].

The selected model for analysis is a neural network setup for the regression case with 5 hidden layers, with 100 nodes each, using the leaky ReLU activation function. Only the first set of subgradients will be found using both *JAX* and backpropagation individually.

TABLE XIV. Differences between gradients computed using JAX automatic differentiation and backpropagation.

Layer	Parameter	Total Error	Maximum Error
1	W	0	0
	b	0	0
2	W	1.055×10^{-15}	5.204×10^{-18}
	b	0	0
3	W	1.079×10^{-15}	5.204×10^{-18}
	b	0	0
4	W	1.043×10^{-15}	4.337×10^{-18}
	b	0	0
5	W	6.881×10^{-16}	4.337×10^{-18}
	b	0	0
6	W	1.692×10^{-16}	1.735×10^{-17}
	b	0	0

Table XIV shows that the two methods agree to machine precision. The total error corresponds to the sum of the individual errors for each weight for each node. Since each node is 100, the total error in each layer is roughly multiplied by that factor. Since the calculation in reality happens at every node, the maximum error in each layer better describes the two methods agreeing to machine precision. There is no error in the bias in any of the layers. Clearly, both methods are capable of finding the same gradient for the same set of layers. This also means that either method can be used to build a neural network and are in that sense equivalent. A further analysis could measure the difference between future gradients, and whether this very tiny difference can cause a drift in the agreement between the two methods after running for many epochs. However, it should be duly noted that with the same learning rate and optimiser method, the parameters should yield an almost identical solution for a reasonable number of epochs.

Concluding that both methods are equivalent, which one is preferable is a topic outside the scope of this paper and this meta-analysis. However, *JAX* does have the capability of just-in-time compilation through "jitting" as can be read in its documentation [11], which can improve computational time. In any case, we did not implement the use of jitting in the report, so this is not a claim that we can safely make.

A final critique-worthy note in the implementation of the two different methods is that during the course of this analysis, it was found that the definition of the MSE cost function in the implementation of automatic differentiation using *JAX* used the full MSE cost function instead of the one described in Equation 8. This is whilst the one for the classification case used the half MSE cost function. This arose due to the division of labour, and, since the two neural network problems were developed and solved independently, the issue was left unnoticed. This was corrected when calculating the gradients specifically for this meta-analysis.

[2] https://github.com/synnesjor/FYS-STK4155/tree/main/Project2

Appendix C: Use of AI/LLM Tools

1. Tools used

- ChatGPT Version GPT-5.6 Luna Reasoning model from Open AI
- ChatGPT Version GPT-6 (Instant) model from OpenAI
- GPT UiO Version GPT-5.6 Terra
- Claude Code Version 2.1.285 Opus 5.5.
- Grammarly free version

2. Code generation and assistance

TABLE XV. Overview of LLM assistance by scripts

File	Level	Description
`utils_regression.py`	2	Function `gradient_func`; Claude wrote the declaration of the cost value. Function `train_network_gradient`; ChatGPT and Claude helped implement optimisers and a short snippet for dictionary for states. Function `train_network_stochastic_gradient`; Claude and ChatGPT helped implement optimisers, and debugged the placements of loops, and implemented convergence check.
`main_regression.py`	2	Part `part_b)`, UiOGPT wrote short snippets for identifying the lowest value MSE and printing correctly. Parts `part_c, part_f, plot_f` and `baseline`; Claude wrote line for dynamic activation function array, ChatGPT wrote snippet to print values to textfile. Additional: throughout script, ChatGPT, Claude and GPTUiO has been used for debuging, changes are implemented by us.
`exercise_d.py`	2	Function `gradient_func`; Claude wrote the declaration of the cost value. Function `train_network_gradient`; ChatGPT and Claude helped implement optimisers and a short snippet for dictionary for states. Function `train_network_stochastic_gradient`; Claude and ChatGPT helped implement optimisers, and debugged the placements of loops, and implemented convergence check.
`main_classification.py`	3	Claude Code was used to generate code to save and load from the different models analysed as well as select a smaller training set from the MNSIT with the same proportions as the full dataset.
`utils_classification.py`	2	The function `feed_forward_batch()` was debugged using ChatGPT and Claude Code. There were some issues with the dictionary used for the activation functions, the issue was found to be how the activation function was passed into the function, either by passing the entirety of the function or calling it by string name.
`heat_map_plot_layers_and_nodes.py`	3	ChatGPT suggested a code skeleton, we filled in the specifics and specified the code so it plottes the desired plots.
`heat_map_plots_cross_validation`	3	ChatGPT suggested a code skeleton, we filled in the specifics and specified the code so it plotted the desired plots.
`meta_analysis.py`	0	—

3. Text writing and editing

TABLE XVI. Overview of LLM assistance for the different sections

Section	Level	Notes
Abstract	1	Grammar check with Grammarly
1. Introduction	1	Grammar check with Grammarly
2. Theory	1	Grammar check with Grammarly
3. Methods	1	Grammar check with Grammarly
4. Results and discussion	3	ChatGPT created overleaf code for tables from our prompts, we edited. Grammar check with Grammarly.
6. Conclusion	1	Grammar check with Grammarly
Appendix A	3	ChatGPT created overleaf code for tables from our prompts, we edited. Grammar check with Grammarly.
Appendix B	3	ChatGPT created overleaf code for tables from our prompts, we edited. Grammar check with Grammarly.
Appendix C	3	ChatGPT created overleaf code for tables from our prompts, we edited. Grammar check with Grammarly.